Programmeren voor fysici

Exam June term 2026

Nick Van Remortel, Hugo Einsle

Guidelines

- This exam assignment counts to 50% of your overall grade, and the other 50% is based on your assignments.
- You may work on your assignments during practice sessions on Thursdays, and, of course, also at home and outside of class in the library.
- Submit your solutions no later than **May 22, 2026**, via Blackboard from your student account.
- Make sure to adhere to the submission criteria and include your name, email, date of submission, as comments in each of the file submitted

Problem setting: Using a Physics Informed Neural Network to solve a PDE

Physics Informed Neural Networks, or PINNs, is a relatively new technique to solve problems described by ordinary or partial differential equations with boundary conditions. The idea got traction in the late 1990's [1, 2] when several authors discussed the idea of by solving these equations with a neural network when incorporating the equation directly into the loss function. During this era, the technique remained a niche academic curiosity because computers weren't fast enough, and modern libraries (like PyTorch or TensorFlow) didn't exist to handle the complex automatic differentiation required.

In this exam exercise, you will develop a Physics Informed Neural Network (PINN) to solve a non-linear partial differential equation (PDE). In a delicate exercise of balance between non-triviality and simplicity, we have chosen the $1+1$-dimensional Burger's equation. It is often called the "poor man's Navier-Stokes" because it contains the exact same mathematical DNA—specifically the competition between convection (moving stuff) and diffusion (smoothing stuff out)—but stripped down to a single dimension to make it solvable without advanced computational tricks and large computing infrastructure. A detailed description of the behavior, solutions and analysis of the Burger's equation is provided in the Appendix. We do recommend you to read this appendix carefully, in order to gain a proper understanding of what you are going to solve/simulate.
In what follows, we will describe the tasks you need to complete for this exam exercise. We will ask you to study an existing Physics Informed Neural Network, in order to grasp the concept, start from a working coded example, and explore the possibilities of this example. Then you will write your own PINN to solve a more interesting or challenging PDE: the Burger's equation. You will study the performance of your implementation in terms of agreement with existing analytical solutions, and in terms of computation time. Given the size of the assignment and the limited amount of students in this year's course, you will have to solve this exam assignment individually. Therefore all rules concerning copying/plagiarism apply: You are allowed to consult your fellow students on their approach, ideas and solutions. You are also encouraged to use AI software to assist you in the coding process, provided that you document properly in your code and in your report, which parts have been produced by AI. Note that AI software is very helpful in giving guidance and solutions, but it is not flawless, in particular for more complex cases. We therefore highly recommend to carefully check code and solutions provided by AI software for bugs, flaws, or outright wrong solutions to the questions you pose.

The final products you need to deliver before the deadline of this project are:

- A PDF with a complete and properly written lab report where you discuss the coding choices, and the computational costs in terms of computing time, memory usage on your personal computing platform. You will describe the solutions to the physics problems posed in this assignment, by means of clear graphs and numbers, and discuss these critically.

- One or more python implementation files where either classes or functions are implemented to solve the problems asked in this assignment. There should also be a main program or demo that produces graphical or numerical output that can be identified in your report.
- If any additional out- or input files are needed to run your program, please add them to your delivered products.

All code should be lint-free, properly documented and written according to the PEP-8 python coding standard. Your code should be easy to understand, extend and re-useable by people who are not the authors of your code. Specify which library versions are used by your code, in your specific conda environment. You are allowed to use your GPU when solving the problems of the assignments, but you are not obliged to do so. Provide in all cases the possibility to exclusively rely on the CPU only.

TASK1: Study and modify an existing PINN implementation

In order to have a simple and understandable starting point, we have provided a working example of a PINN that solves a simple linear second order ordinary differential equation (ODE). The ODE that is solved here is that of the movement of a mass on a spring, including dampening or friction, described by the following equation:

$$m\frac{d^2y}{dt^2} + c\frac{dy}{dt} + ky = 0, \tag{1}$$

If no external Force is applied, the equation is homogeneous. The analytical solution to this problem is well known and is of the form:

$$y(t) = Ae^{r_1 t} + Be^{r_2 t}, \tag{2}$$

with $r_{1,2}$ given by

$$r_{1,2} = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m}. \tag{3}$$

Depending on the value of the discriminant $c^2 - 4mk$, there are three distinct solutions:

- Overdamped: $c^2 > 4mk \Rightarrow y(t) = Ae^{r_1 t} + Be^{r_2 t}$
- Critically damped: $c^2 = 4mk \Rightarrow y(t) = (A + Bt)e^{-\omega_0 t}$
- Underdamped: $c^2 < 4mk \Rightarrow y(t) = e^{-\frac{c}{2m}t}(A\cos\omega_d t + B\sin\omega_d t)$

Note that for the critically damped case, there is only one value for r. In order to keep the two solutions linearly independent, the second solution is multiplied by t. Oscillatory behavior can be observed in the underdamped case. In the frictionless case, the natural frequency equals $\omega_0 = \sqrt{\frac{k}{m}}$, but when friction is applied, the oscillation frequency is always lower: $\omega_d = \omega_0\sqrt{1 - \frac{c^2}{4km}}$. It is important to note two important properties of this equation:

- The equation has an infinite amount of solutions, even when all parameters m, k, and c are fixed. There exist only one unique solution when two boundary conditions are given, in this case they are starting conditions: (a) the initial amplitude, given by $y(t = 0)$, and (b) the initial velocity, given by $y'(t = 0)$.
- An infinite number of combinations of values of the parameters m, k, and c can yield the same solution. In other words, from observing the solution, one can not unambiguously determine the values of the parameters m, k, and c. Only the ratios c/m, and k/m are relevant for the treatment of this problem, or if preferred, one can substitute both with the natural frequency ω_0, and the damping ratio $\zeta = \frac{c}{2\sqrt{mk}}$

The latter can be deduced from rewriting the original equation in 1:

$$\frac{d^2y}{dt^2} + \frac{c}{m}\frac{dy}{dt} + \frac{k}{m}y = 0, \tag{4}$$

A working example of a PINN that solves equation 1 can be obtained from the GitHub repository: `https://github.com/remortel/ADVANCED_PROGRAMMING/tree/main/PINN`. There are two versions of the program: (a) `pinn_spring_damper.py`, and (b) `pinn_spring_damper_improved.py`. Each are produced with AI assistance. The codes are working reasonably well, and produce acceptable results, but are far from optimal in many aspects: there is linting, they are not in PEP8 standard, they are not very re-useable, or extendable, and some

of the program logic is very sub-optimal in terms of performance. The two codes differ in one crucial aspect, the two initial conditions $y(0)$ and $y'(0)$ are included in the loss function of the improved version (b), while they are omitted in solution (a).

Task 1.1: Understanding an existing PINN example (20 points)

For this task, we will ask you a couple of questions, which you simply have to answer in your report. It will demonstrate that you have understood the method and the example code and it should allow you to make suggestions to improve the code structure and performance.

Questions on the method:

1. Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.
2. Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?
3. The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?
4. What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?
5. How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?
6. The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?
7. The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is is a necessary ingredient for training a neural network? What are its main hyper-parameters? Are there alternatives that can be usefully applied in this example?
8. How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?
9. Would it make sense to have a set of validation points? How would these look like? How would you use them?

Questions on the code:

1. Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

2. What is the purpose of the

`np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

3. The data points are created randomly and then sorted before being offered to the neural net, using this statement

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

4. The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

5. What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

6. What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()
optimiser.step()
scheduler.step()
```

7. Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)
y_obs_t = to_tensor(y_obs)
t_col_t = to_tensor(t_col, requires_grad=True)
t_ic_t  = to_tensor(t_ic,  requires_grad=True)
```

8. When presenting arrays of time points to the input of my neural network, I need to convert them into a pytorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
            requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the torch.tensor class do, and why do I need to do this?

9. When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element `[0]` of this output in the statements below:

```
dy = torch.autograd.grad(
y_hat, t,
grad_outputs=torch.ones_like(y_hat),
create_graph=True)[0]
```

Task 1.2: Cleaning and Optimizing the example PINN (20 points)

Given that you completely understand the structure, flaws and necessities of the example code, clean it up, make it compatible with the PEP-8 standards, make improvements to the algorithm, where you think it is useful. Check the performance of your model by means of an additional set of collocation points that you only use for validation and not for training. Put the performance plots and the results in your report and explain why your improved version is better than the example version.

Change some of the most relevant or impactful hyperparameters of your model and determine which model with which hyperparameters performs the best. Provide us with the code of this improved PINN model for the damped spring, and paste the outputs in your report with a short discussion of what has changed and why your model is better than the example.

TIP: Save your output before plotting to one or more output files in a format of your choice. In this way you will not need to rerun your PINN training when you want to expand or modify your graphical output. It can also be useful to write a reusable plotting code in a separate python program, since you will produce several times the same set of plots.

Task 1.3: Solving the Inverse problem (20 points)

Solving inverse problems—where you use data to "discover" unknown physical parameters—is one of the most powerful applications of PINNs. While a standard neural network only learns the mapping $t \rightarrow y(t)$, a PINN can simultaneously learn the mapping and the constants that govern the underlying physics. In this case, you can treat ζ and ω_0 as learnable parameters (variables) rather than fixed constants.

The network can be defined as before. You just provide more data points generated according to an underlying ground truth for ζ and ω_0. In an initial phase you can generate this data with the exact values as predicted by the analytical solution. This would be a sanity check to determine whether your modified PINN is able to recover exactly the underlying values for these parameters. In a second stage, you will add noise to the data points, similar to what was done for the data points in the old example. You can simply add white or Gaussian noise, or you can make certain points more noisy than others. In the previous example, only 15 noisy data points were provided. For the determination of the ζ and ω_0 parameters, it is recommended to use some more data points (mimimally 40).

In order to implement the inverse PINN you essentially need to change two things to your previous task.

1. Rewrite the physics loss in function of the two unknown parameters

The original ODE from eq. 1, re-written in the parameters ζ and ω_0 is the following:

$$\hat{y}'' + 2\zeta\omega_0\hat{y}' + \omega_0^2\hat{y} = 0 \tag{5}$$

2. Add two additional parameters to your Pytorch model

In the definition of your `nn.model`, add simply these lines at the end of your `__init__` method:

```
        self.zeta_hat = nn.Parameter(torch.tensor([0.01], requires_grad=True))
        self.w0_hat = nn.Parameter(torch.tensor([1.0], requires_grad=True))
```

We call the parameters to be estimated `zeta_hat` and `w0_hat` in order to avoid confusion with the underlying truth values `zeta` and `w0`.

Indeed, rather than rethinking your whole approach in function of different inputs and outputs of your neural net, you simply keep the neural net as it is, but in the implementation of your model, you add 2 extra parameters to be optimized, in addition to the few thousands you were already optimizing before!

3. Add two additional parameters to your optimizer

In the optimizer object, add these two parameters, and if desired give them a different learning rate than the parameters of your neural network model:

```
optimizer = torch.optim.Adam([
    {'params': model.net.parameters(), 'lr': 1e-3},
    {'params': [model.zeta_hat, model.w0_hat], 'lr': 1e-2}
])
```

After training, you can get the optimized values of your parameters via

```
    model.zeta_hat.item()
    model.w0_hat.item()
```

Given these tips, adopt your earlier program, to allow for the option of the determination of these two physical parameters of the model. Note that now the contribution of the data to the total loss function is much more important than that of the physics loss, even though the physics loss contains your two unknown physics parameters. You can play with the individual contributions of data and physics loss to investigate the effect on the determined physics parameters. Also note that in principle you do not need the loss contribution from the initial condition anymore, provided that you have enough data points to implicitly constrain it. In some cases, you might not know the initial condition and then you are forced to omit it. In case you know the initial condition, it is always better to add it to the loss function, since it will provide a big constraint on the possible solutions and will ensure a much more stable estimate of your physics parameters.

The scenarios you have to finally investigate in this task are:

1. Demonstrate that you are able to recover the ζ and ω_0 parameters for an underdamped case.
2. Demonstrate that you are able to recover the ζ and ω_0 parameters for a critically damped case. Note that this will be more difficult.
3. Demonstrate that you are able to recover the ζ and ω_0 parameters for a critically damped case. Here you absolutely need to add the initial condition to your loss function, since this will constrain the values of your parameters most.

Also, discuss the following question in your report: What sources of uncertainty are propagated into your estimates of the physical parameters ζ and ω_0? What techniques could you apply to estimate these uncertainties?

TASK2: Develop a PINN that solves the 1+1 D Burger's equation

We are now stepping up in terms of complexity of the problem to be solved. Rather than solving a linear and homogeneous 2nd order ODE, we are now going to solve a non-linear, 2nd order PDE in the form of the viscid 1D Burger's equation. More information on the history, the analysis, and the applications of Burger's equation is provided in the appendix of this assignment. There is also a nice review paper on it [3]. The bibiographical references are just for background, but we do recommend you to read the appendix thoroughly to understand the behavior of Burger's equation. For the task at hand, it is sufficient to repeat the analytical expression of the Burger's equation:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = 0. \tag{6}$$

It is a 1-D problem, in the sense that we are now solving a scalar field $u(x)$ that depends on a 1-D position x. But since there is also a time dependence, it can also be considered as a 1+1-D problem. The solutions are curves of $u(x,t)$ in the 2-D (x,t) plane, but more conveniently we present the solutions as 1D $u(x)$ curves at various discrete and well chosen times, or even as an animated plot of $u(x)$ through time.
For some cases, this equation has analytical solutions, and for other cases only numerical approximations exist. The solutions to the equation have the tendency to produce shock fronts or shock waves, that manifest as an abrupt or discontinuous behavior of the solution at longer time scales, as is described in the appendix. The emergence of a shock front will depend on the initial conditions connected to the problem. Whether such as shock front will form and at what approximate time scale can be determined a-priori via a nifty method known as the method of characteristic curves. This method is also demonstrated in the appendix.
It should be intuitively clear that discontinuities in the solutions are hard to model, and should pose some challenges to our PINN implementation of this problem. The inclusion of the viscid contribution to the equation via the term $-\nu\frac{\partial^2 u}{\partial x^2}$ should smoothen out the shock front. The biggest challenge will therefore be in lowering the kinematic viscosity parameter, ν from relatively high values to zero. In this problem, we will need to modify the architecture of our previous NNet model slightly, in order to accept two input features: time, t, and space, x, while we still have one output quantity u. Your tasks consist therefore of the following:

Task 2.1: Solving the Viscid Burger's equation with a PINN (20 points)

Implement a new python program that solves the 1D burger's equation using a PINN that has 2 input features and one output feature. The amount of hidden layers and the amount of nodes per layer is up to you to choose and optimize. You do not provide data points or data constraints for this particular case, but since you will have to solve the inverse problem to determine the viscosity, given a certain amount of input data, it is good to keep that option open. The physics loss is determined by the expression given in eq. 6. The most important contribution to the loss function comes from the initial condition. The initial condition starts with a profile of $u(x)$ at time $t = 0$. Several interesting cases exist, with known analytical solutions, for a step upwards, a step downwards, a linear ramp up and a linear ramp down, a truncated sine, a delta function, a gaussian function, an N-shaped (or sawtooth) curve etc. You can see some of the solutions for these initial conditions on the following wikipedia page: `https://en.wikipedia.org/wiki/Burgers%27_equation`. We want you to solve the equation for at least three initial conditions:

1. A Gaussian initial condition: $u(x,0) = e^{-x^2/2}$
2. A step up function: $u(x,0) = 1,$ when $x > 0.$, and zero elsewhere
3. An N-wave: $u(x,0) = -x,$ when $|x| < 1.$, and zero elsewhere

Provide the solutions for these three initial conditions for at least three chosen times: (a) time $t = 0$, (b) a time in between zero and the possible formation of a shock front, (c) a time close or at the time when a shock front is formed. You can determine whether a shock front is formed and wt what time, by looking at the method of characteristics, demonstrated in the appendix. You should also show these solutions for three values of the kinematic viscosity: (1) $\nu = 0.1$, (2) $\nu = 0.05$, and (3) $\nu = 0.01$.

TIPS: Choose your collocation points randomly in time and space: 200 randomly chosen (x, t) pairs is more than enough. Providing a fixed grid of, say 100×100, space-time points is not a good idea, since it will generate typically a too large number of collocation points, and due to their equal spacing, the network can fine-tune on that specific spacing. The initial condition should be well defined, so you should typically generate some more points for the initial condition profile $u(x, 0)$. For all initial conditions described above, analytical solutions exist, so you can always compare your output with the analytical solution.

Task 2.2: Solving the Viscid Burger's equation with a PINN (20 points)

As you might have expected, it is also in this case perfectly possible to determine the one physical parameter of the Burger's equation: the kinematic viscosity ν. The implementation is exactly the same as in the earlier scenario: add a parameter to your torch model, generate a sufficient amount of artificial noisy data points from an analytically known solution (preferably one that produces a shock front), and let your optimizer find the optimized value. Demonstrate the recovery of the viscosity parameter for two scenario's, one with a high value, say $\nu = 0.1$, and one with a low value, say $\nu = 0.01$.

References

[1] M. W. M. G. Dissanayake and M. Phan-Thien, "Neural-Network-Based Approximations for solving Partial Differential Equations", COMMUNICATIONS IN NUMERICAL METHODS IN ENGINEERING, Vol.10, 195-201 (1994)

[2] I. E. Lagaris, A. Likas and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," in IEEE Transactions on Neural Networks, vol. 9, no. 5, pp. 987-1000, Sept. 1998, doi: 10.1109/72.712178.

[3] N. K. Kumar, "A REVIEW ON BURGERS' EQUATIONS AND IT'S APPLICATIONS", Journal of Institute of Science and Technology, 28(2), 49-52 (2023), `https://doi.org/10.3126/jist.v28i2.61073`

Appendix: Burger's equation

Introduction: The form and meaning of the 1+1 D Burger's equation

Burgers' equation or Bateman-Burgers equation is a fundamental partial differential equation (PDE) occurring in various areas of applied mathematics, such as fluid mechanics, nonlinear acoustics, gas dynamics, and traffic flow.

We may obtain Burgers's equation as a simplified version of the Navier-Stokes equations. The basic equations that govern the viscid flow of simple substances like air and water under normal conditions are called the Navier-Stokes equations (nonlinear system of PDEs):

$$\rho(\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u}) = -\nabla p + \mu \Delta \mathbf{u} + \mathbf{F}, \tag{7}$$

where $\mathbf{u}$ is the velocity of a fluid, p the fluid pressure, μ the fluid dynamic viscosity, and $\mathbf{F}$ and external force. The kinematic viscosity equals $\nu = \mu/\rho$. The notation $\mathbf{u}_t$ corresponds to the partial derivative of $\mathbf{u}$ with respect to time, $\partial \mathbf{u}/\partial t$ and $\Delta \mathbf{u}$ represents the laplacian of $\mathbf{u}$, ie. $\Delta \mathbf{u} = \nabla^2 \mathbf{u} = \partial^2 \mathbf{u}/\partial x^2 + \partial^2 \mathbf{u}/\partial y^2 + \partial^2 \mathbf{u}/\partial z^2$. The term $\rho(\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u})$ corresponds to the inertial forces, $-\nabla p$ corresponds to pressure forces, and $\mu \Delta \mathbf{u}$ corresponds to viscid forces.
The Navier-Stokes equations were derived by Navier, Poisson, Saint-Venant, and Stokes between 1827 and 1845. These equations are always solved together with the continuity equation $\rho_t + \nabla \cdot (\rho \mathbf{u}) = 0$. For incompressible flows (density of the fluid is constant), the continuity equation yields $\nabla \cdot \mathbf{u} = 0$. As such, the Navier-Stokes equations represent the conservation of momentum, while the continuity equation represents the conservation of mass.

If we reduce the problem to only one space and one time dimension $(1+1)$-D, drop the pressure term from the Navier-Stokes equations in eq. 7 and assume no external force, we obtain the most commonly known form of the $(1+1)$-D Bateman-Burgers equation:

$$u_t + u \cdot u_x = \nu u_{xx}, \tag{8}$$

or in a more common notation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}. \tag{9}$$

An even further simplification arises when we assume the viscosity is zero. Then we obtain the inviscid Burger's equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0. \tag{10}$$

A specific solution of the Burger's equation always depends on the initial condition of the field: $u(x, t = 0) = f(x)$, and/or boundary conditions of the solution domain $x \in [x_L; x_R]$: $u(x = x_R) = A$; $u(x = x_R) = B$.

Interpretation and solutions

The three terms in the viscid Burger's equation each have a specific physical interpretation:

The term $\frac{\partial u}{\partial t}$ expresses the rate of change of the field $u(x)$ at any position x over time, it implies unstationary (moving) solutions.

The term $u\frac{\partial u}{\partial x}$ is an advection term, which expresses the transport of a quantity along with the flow. Here, the transported quantity is the value of the field $u(x)$, with a speed that is proportional to the field itself! This makes the equation non-linear. This means that regions exhibiting large values of u will be propagated rightwards quicker than regions exhibiting smaller values of u; in other words, if u is decreasing in the x-direction, initially, then larger u's that lie in the backside will catch up with smaller u's on the front side. This is the mathematical equivalent of a wave where the top of the wave travels faster than the bottom, causing it to "fold" over itself. The presence of this term implies that if there exists an initial spatial gradient in the field $u(x)$, that either a shock-front, a region with ever increasing gradient $u\frac{\partial u}{\partial x}$, will develop over time, or a rarefaction, a region with ever decreasing gradient, will develop. We will demonstrate both behavior in what follows.

The term $\nu\frac{\partial^2 u}{\partial x^2}$ is a viscous or dissipative or diffusion term. It resists the formation of sharp gradients and represents the internal friction. The kinematic viscosity ν expresses the "thickness" of the fluid. Large ν makes the solution blurry or smoothened out, while a tiny ν allows for sharp, violent shocks. A "stationary" shock profile forms where the steepening caused by the advection term is perfectly balanced by the smoothing caused by the viscid term.

The interpretation of The 1-D field variable $u(x)$ depends on the physical problem: In the context of fluid dynamics or a traffic simulation, it represents the flow velocity; in the context of acoustics, it represents pressure or sound amplitude; in the context of material growth, it represents the surface height profile, $\cdots$

The initial conditions will determine whether the solutions of the Burger's equation will develop shock phenomena or not.

The method of characteristics

The solutions can be analyzed using a technique called the Characteristic Method, or Method Of Characteristics (MOC).
The method of characteristics is a powerful technique for solving first-order partial differential equations (PDEs) by reducing them to a set of coupled ordinary differential equations (ODEs). By finding special curves—called characteristic curves—the PDE becomes an ODE along these paths, making the solution easier to find. The technique is widely used in mathematical physics and can be developed quite generally. We limit ourselves here to the core concept in the context of our Burger's equation:
Consider a PDE of the form:

$$a(u,x,t)\frac{\partial u}{\partial t} + b(u,x,t)\frac{\partial u}{\partial x} = c(u,x,t). \tag{11}$$

Suppose we are looking for a curve $x(s), t(s)$, along which we want to track the solution $u(x,t)$. The chain rule tells us that:

$$\frac{du}{ds} = \frac{\partial u}{\partial t}\frac{dt}{ds} + \frac{\partial u}{\partial x}\frac{dx}{ds}, \tag{12}$$

then we see through identification with eq. 11 that we obtain a set of coupled ODE's of the form:

$$\frac{dt}{ds} = a(u,x,t) \tag{13}$$
$$\frac{dx}{ds} = b(u,x,t) \tag{14}$$
$$\frac{du}{ds} = c(u,x,t) \tag{15}$$

If we apply this to our inviscid Burger's equation from eq. 10, we see that these PDE's become very simple:

$$\frac{dt}{ds} = 1 \rightarrow s = t \tag{16}$$
$$\frac{dx}{ds} = u(x,t) \rightarrow \frac{dx}{dt} = u(x,t) \tag{17}$$
$$\frac{du}{ds} = 0 \rightarrow u(x,t) = u(x,t=0) = u_0(x) \tag{18}$$

This implies that along these characteristic trajectories u is constant and equal to the starting condition $u(x,t) = u_0(x)$ and that the trajectories themselves are straight lines $x(t) = u_0(x)t + x_0$. The trajectories for 25 discrete space points x along which $u(x)$ remains constant over time is shown in Fig. 1 for an initial condition $u_0(x) = c$. The initial starting points $u(x_i, t=0)$, are marked by the red dots. Because $u(x)$ is initially constant over all positions x, we see that the value of the field is transported linearly in time. If u represents a velocity field, the entire fluid domain moves to the right (if $c > 0$) or left (if $c < 0$) with constant speed, with no internal distortion or wave breaking.

The situation changes when one applies different initial conditions: Suppose the profile of $u(x, t=0)$ is of the following form:

$$f(x) = \begin{cases} 1, & \text{if } x < 0.25 \\ -2x + 1.5, & \text{if } 0.25 \leq x \leq 0.75 \\ 0, & \text{if } x > 0.75 \end{cases}$$

When using this initial condition, the solution can be analytically calculated and is shown in Fig. 2. The solution develops over time into a discontinuous shock profile at $x = 0.75$ and time $t = 0.5$ which is predicted by the trajectories shown in Fig. 3, that start to intersect exactly at $(x,t) = (0.75, 0.5)$. A rarefaction occurs if the

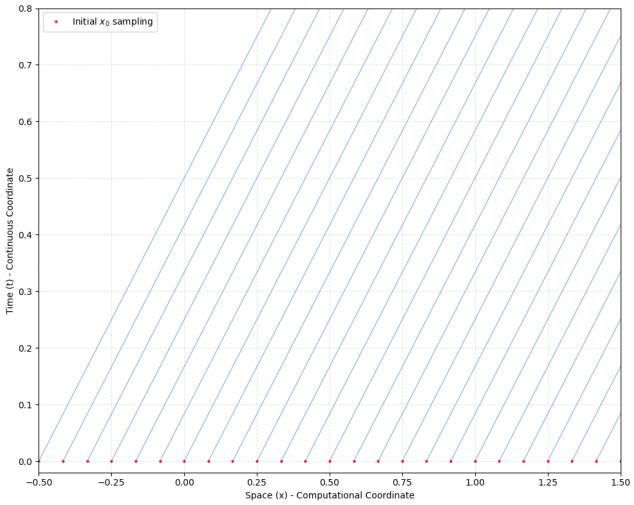

Figure 1: Trajectories of constant field transport for the inviscid Burger's equations with a uniform and constant initial velocity field.

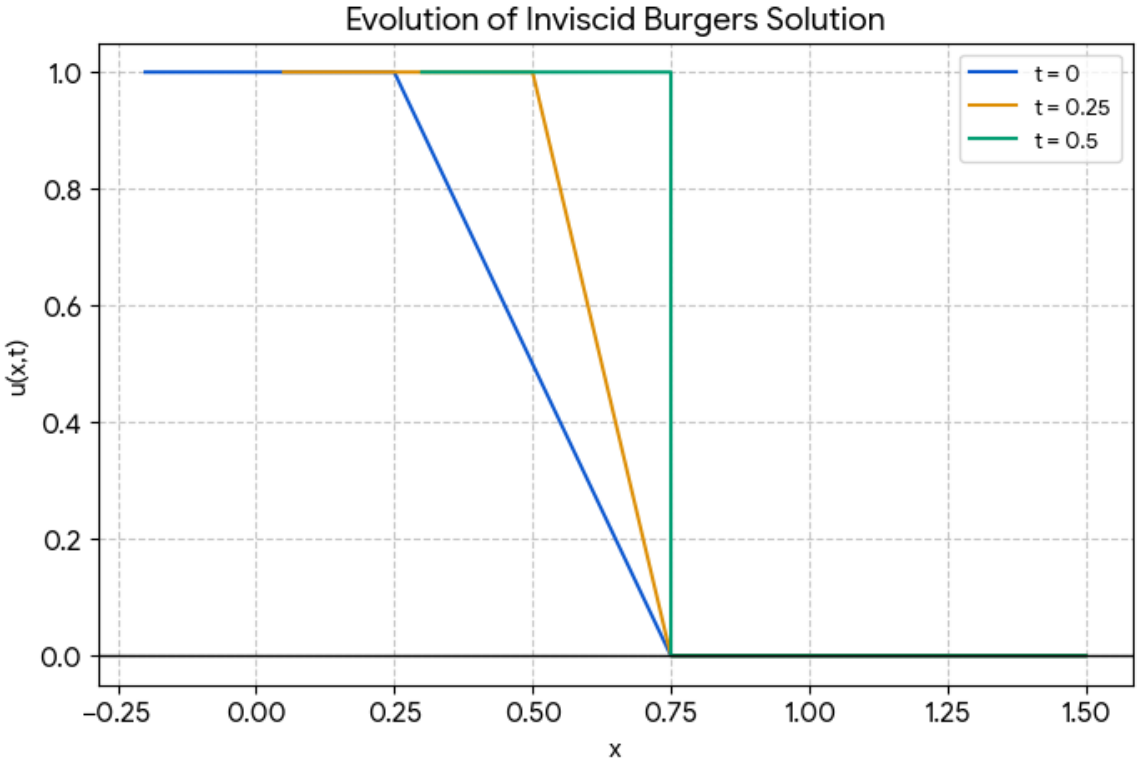

Figure 2: The time evolution of the field $u(x, t)$. showing the development of a discontinuous shock front at late times t.

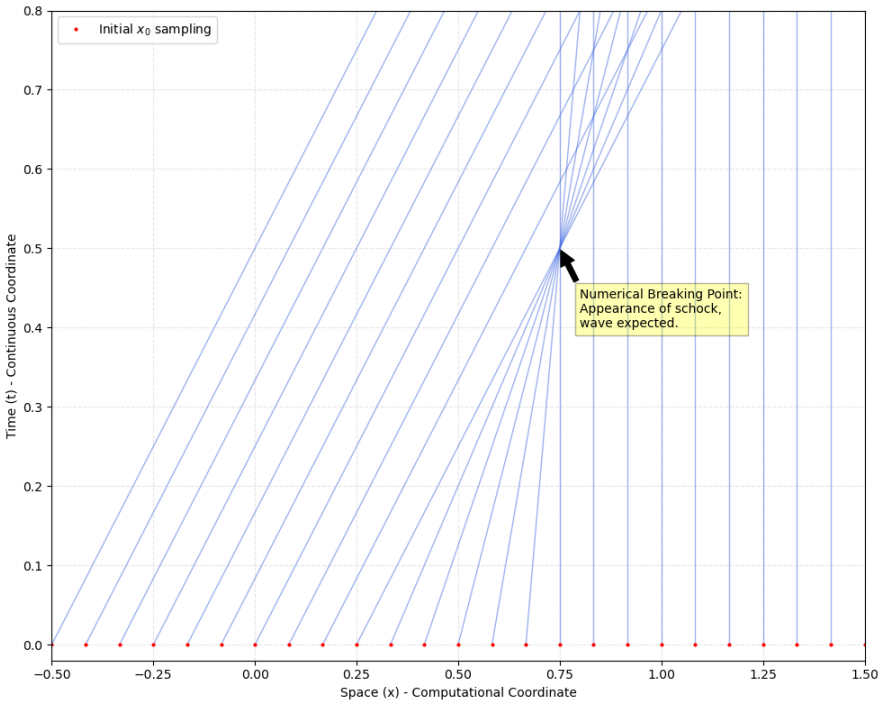

Figure 3: The trajectories of constant field solutions change slope for large initial x_0 due to the initial conditions $u_0(x, t = 0)$ shown in Fig 2. The position in $x = 0.75$ where the trajectories cross indicate that several solutions of $u(x, t)$ exist at this point, implying a numerical complication due to the existence of a shock wave.

initial condition shows a low value of the field $u(x)$ at low values of x and a high value at large values of x. This corresponds, for example to the initial condition:

$$f(x) = \begin{cases} 0, & \text{if } x < 0.25 \\ 1, & \text{if } x > 0.25 \end{cases}$$

The time evolution of $u(x, t)$ in the rarefaction case is shown in Fig. 4, and the characteristic curves are shown to fan out this case, as can be seen in Fig. 5.

The impact of viscosity

For the viscid (viscous) Burgers' equation, the method of characteristics works differently than for the inviscid version. In the inviscid case, the solution $u(x, t)$ is constant along straight characteristic lines. However, the presence of the viscosity term $(\nu \frac{\partial^2 u}{\partial x^2})$ means that $u(x, t)$ is no longer constant along those lines—it diffuses. We will not go into the details of the characteristic trajectories for these cases, but one can intuitively understand the behavior as such:

The path or trajectory equation maintains the general form: $\frac{dx}{dt} = u(x, t)$, while the evolution equation is now the diffusion equation: $\frac{du}{dt} = \nu \frac{\partial^2 u}{\partial x^2}$. The characteristic curves are not straight lines anymore; they become curved due to the diffusion term. As an illustration, in Fig. 6 we show the characteristic curves for the same initial condition as in Fig. 2, and in Fig. 7, the solution at a few time steps and for various kinematic viscosities, ν.

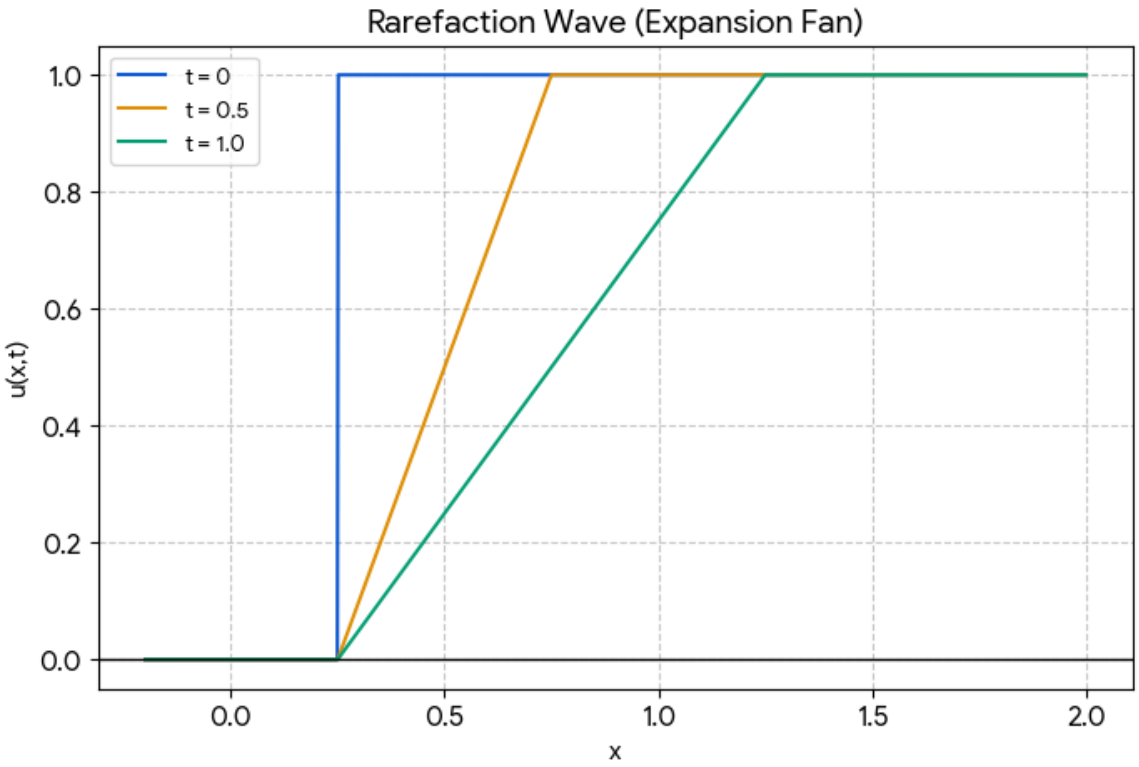

Figure 4: The time evolution of the field $u(x, t)$. showing the development of a rarefaction manifesting as a diminishing spatial gradient at late times t.

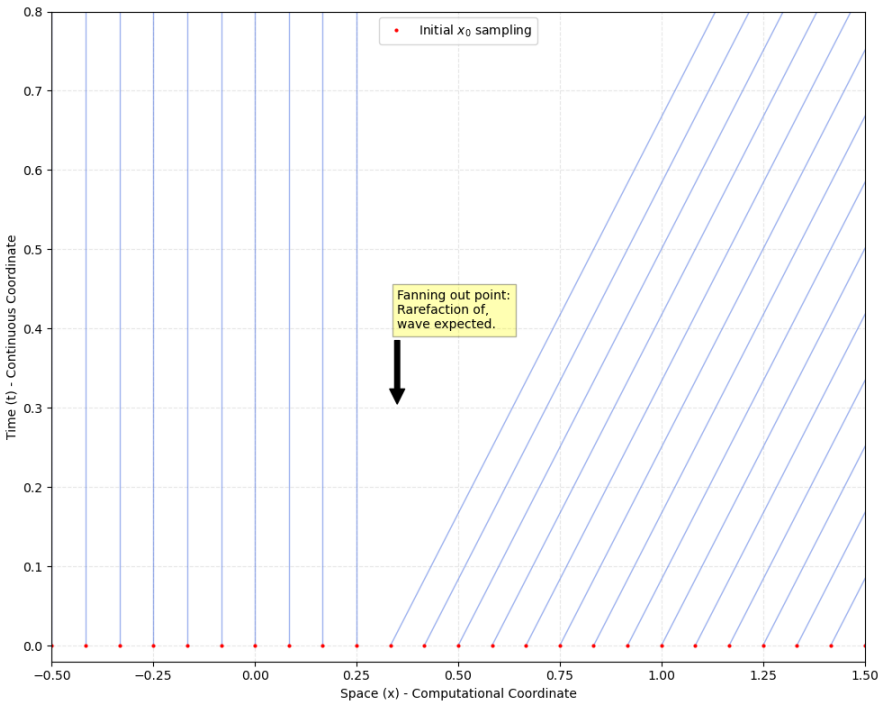

Figure 5: The trajectories of constant field solutions change slope for large initial x_0 due to the initial conditions $u_0(x, t = 0)$ shown in Fig. 4. The position in $x = 0.25$ indicates the rarefaction where the trajectories start to fan out.

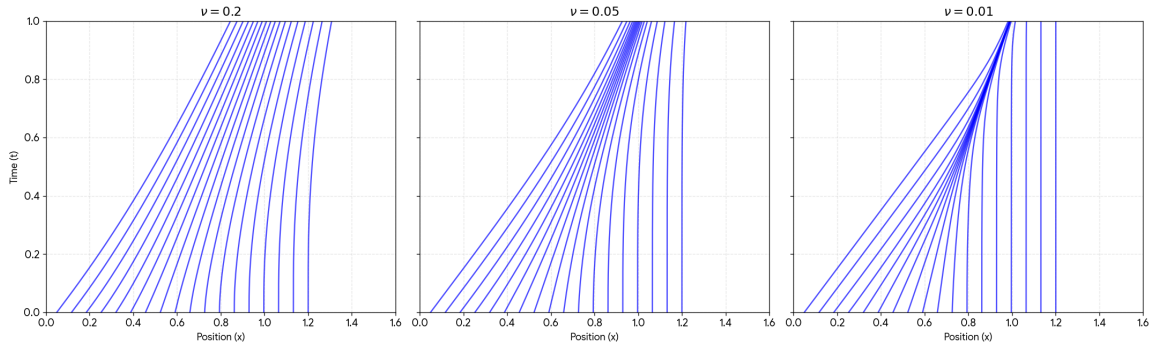

Figure 6: The trajectories of constant field solutions for the viscid Burger's equation with a ramp down initial condition, for various values of the kinematic viscosity, ν.

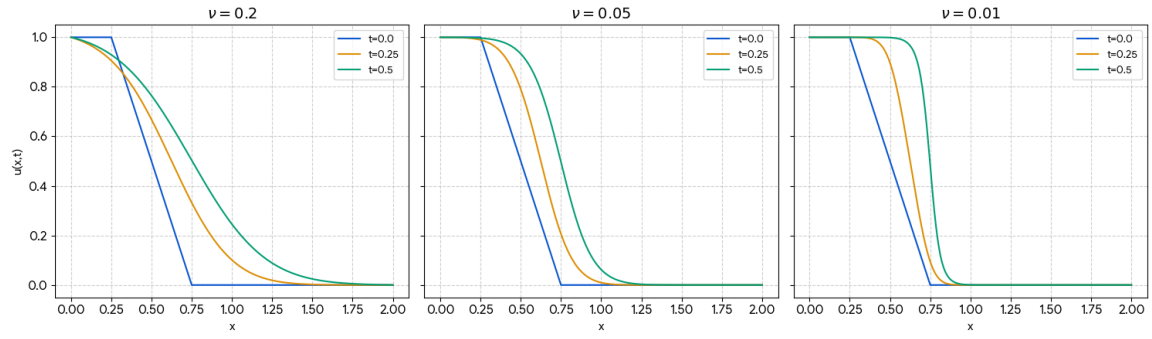

Figure 7: The time evolution of the field $u(x, t)$. showing the development of a smoothened shock front at late times t, for various values of the kinematic viscosity, ν.